

Sequenzanalyse 2 - Abgabe 2

Laura Häger und Liliana Sanfilippo

May 10, 2026

1 Aufgabe 1

1.1 Wir arbeiten in dieser Aufgabe mit den Reads in `/prj/seqan/Assembly/reads.fq`, die im fastq-Format vorliegen

Informiere dich über dieses Format Okay.

Erkläre kurz die Unterschiede zum fasta-Format Das fastq-Format basiert auf dem fasta-Format. Es beinhaltet mehr Informationen über die Sequenz. Diese Informationen werden durch weitere Zeilen dargestellt, so dass jede Sequenz vier Zeilen hat, statt nur eine wie in einer fasta-Datei. Die vier Zeilen sind einmal die Identifizierungszeile, die Sequenz selbst, die Identifizierungszeile mit einem Plus davor und ein Qualitäts-Score für jede Base bzw. Position der Sequenz (*associated per base quality score*). [2, 3]

1.2 Wir werden nun (compacted bi-directed) De Bruijn (Sub-) Graphen für $k = 11, 15, 17, 21, 31$ mit `ggcat` erzeugen. Um diese Graphen später mit `Bandage` visualisieren zu können, brauchen wir sie im GFA-Format

Gemacht mit:

```
ggcat build --gfa-v1 --kmer-length 11 reads.fq -o output11.gfa
```

```
ggcat build --gfa-v1 --kmer-length 15 reads.fq -o output15.gfa
```

```
ggcat build --gfa-v1 --kmer-length 17 reads.fq -o output17.gfa
```

```
ggcat build --gfa-v1 --kmer-length 21 reads.fq -o output21.gfa
```

```
ggcat build --gfa-v1 --kmer-length 31 reads.fq -o output31.gfa
```

Schaue dir außerdem die Kommandozeilenparameter `-e` und `-s` an. `-e` ist ein Output-Modus und wird benutzt für “build links between maximal unitigs”, also es erstellt die Kanten zwischen allen Knoten. `-s` setzt die “Minimum multiplicity required to keep a kmer”, also wie oft ein kmer auftauchen muss, damit es einbezogen wird. [1]

Wie musst du sie setzen, um einen De Bruijn Graphen mit Kanten und von allen in den Reads enthaltenen k-meren zu erzeugen? Für `-s` muss man als `-s 1` verwenden, damit jedes kmer behalten/verwendet wird und `-e` kann man einfach so benutzen, da `-generate-maximal-unitigs-links` der default ist und wir genau das wollen.

Gemacht in der Art:

```
ggcat build --gfa-v1 --kmer-length 11 -s 1 -e reads.fq -o output11.gfa
```

.
. .
.

1.3 Visualisiere die erzielten Ergebnisse mit Bandage und bilde sie in deiner Abgabe ab

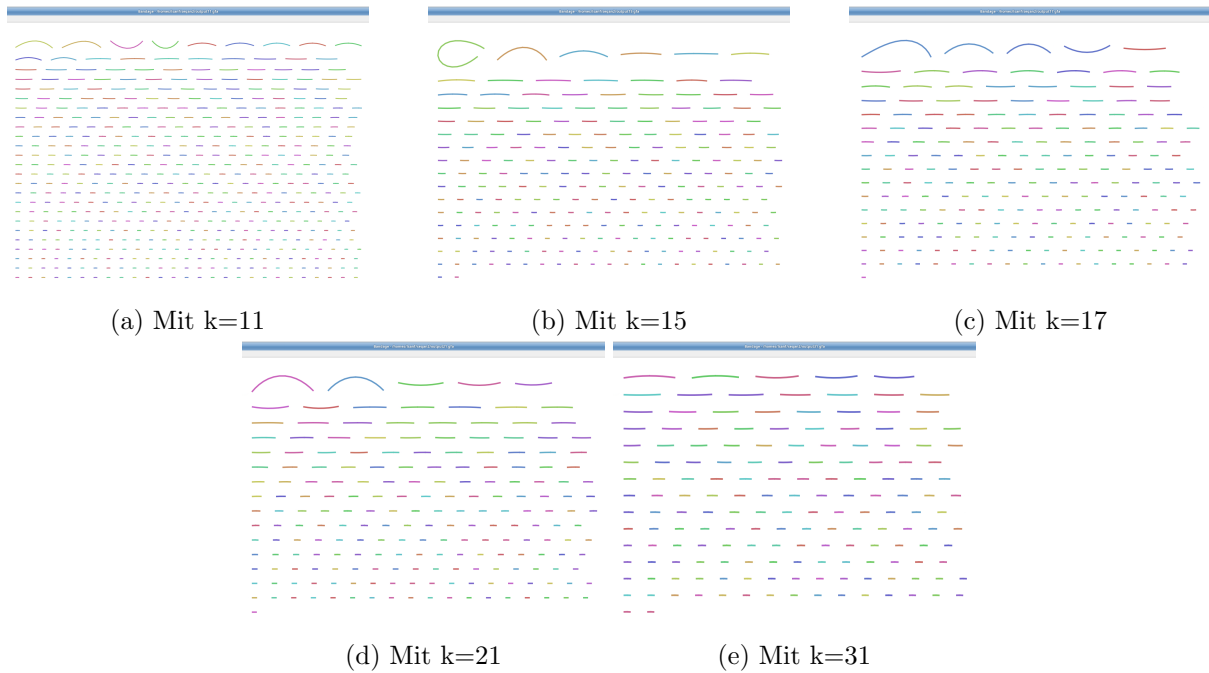


Figure 1: Hier kann man ganz wunderbar sehen, was passiert, wenn man "-s 1 -e" nicht anwendet.

Nun aber Schluss mit lustig, hier die tatsächlichen Visualisierungen:

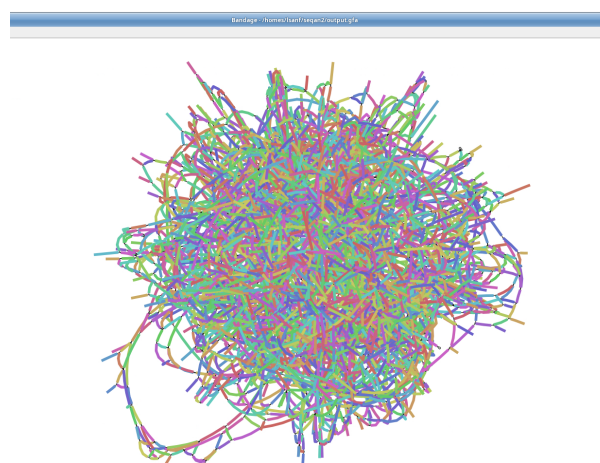
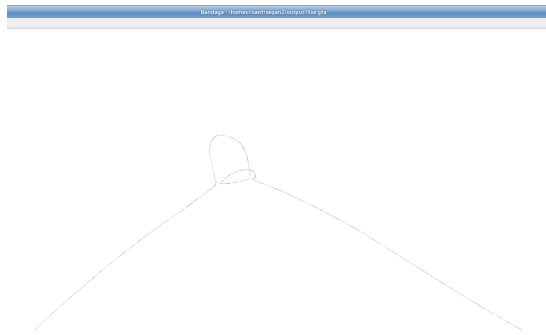
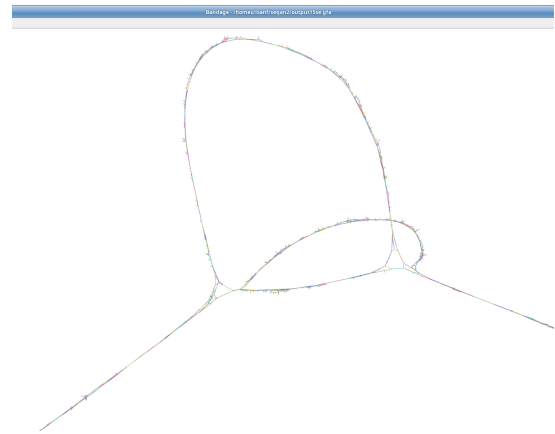


Figure 2: Mit k=11

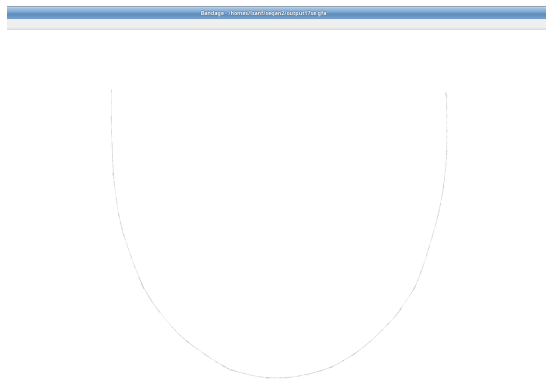


(a) Übersicht

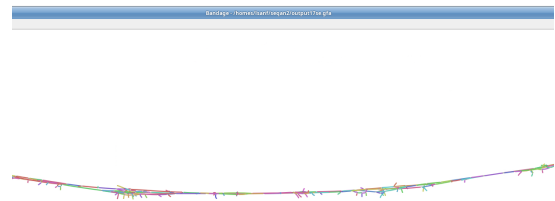


(b) Zoom ca. 20%

Figure 3: Mit $k=15$



(a) Übersicht



(b) Zoom ca. 50%

Figure 4: Mit $k=17$



Figure 5: Mit $k = 31$

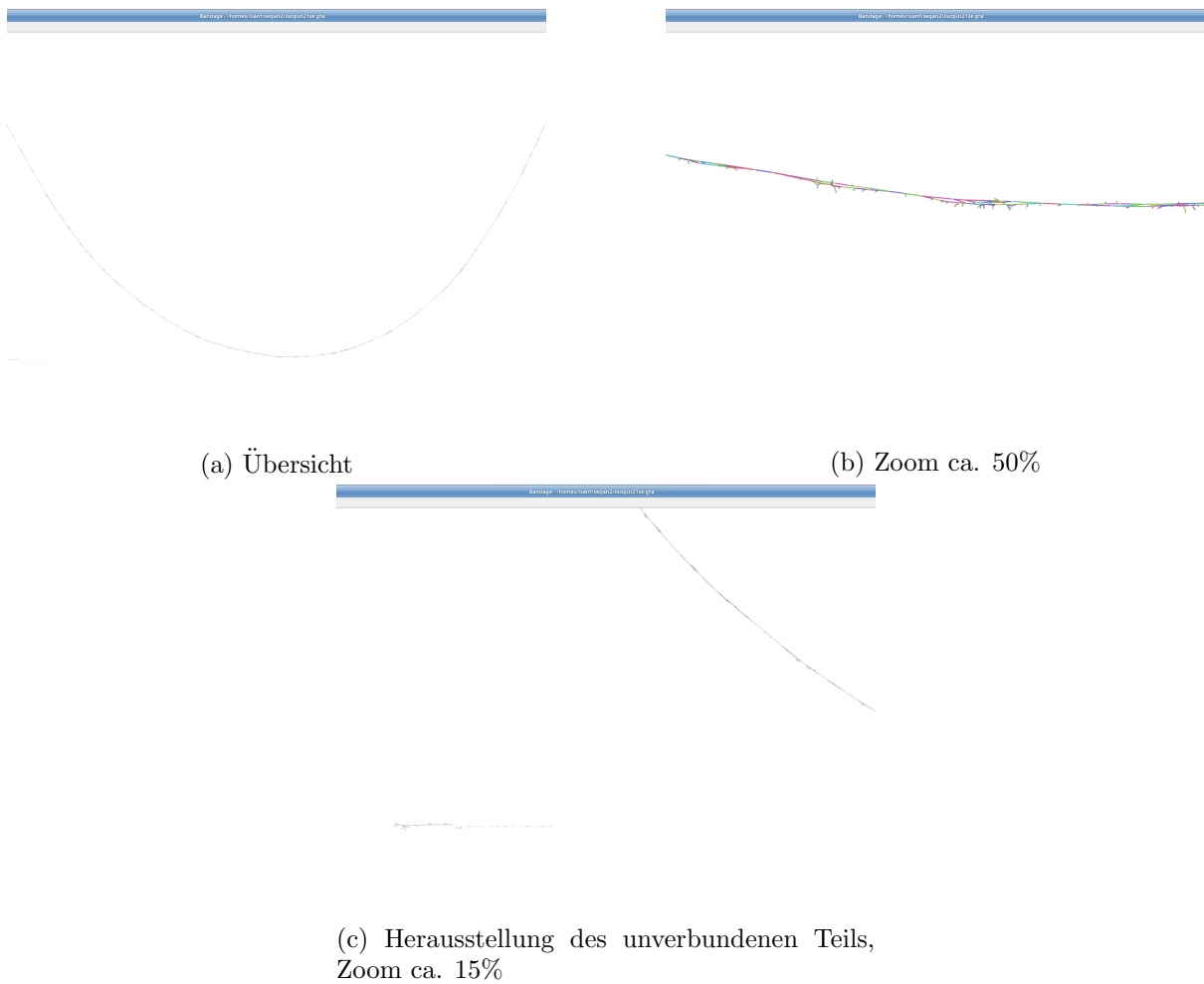


Figure 6: Mit $k=21$

Wie unterscheiden sich die erzeugten Graphen? Je kleiner das k ist, desto verzweigter, aber auch kompakter, ist der Graph und desto mehr Kanten gibt es insgesamt. Die Verzweigungen sind bilden auch kleinere Schleifen bei kleinen k .

Bei größeren k ist der Graph nicht mehr vollständig zusammenhängend.

Erkläre kurz in Bezug auf k wie die Unterschiede zustandekommen. Bei kleineren k s sind die k -mere kürzer und daher kommen sie öfter vor und es müssen viele Kanten zwischen den Knoten gebildet werden. Daher sind die Graphen kompakt, aber unübersichtlich. Es ist sozusagen alles recht nah aneinander, aber auch nicht sonderlich aussagekräftig.

Wählt man größere k , so werden die Schleifen und Verzweigungen (visuell) größer, da es weniger Verbindungen zwischen k -meren gibt und nicht mehr alles ein Knäuel ist, sondern sich die tatsächlichen Sequenzen besser herauskristallisieren.

Ab einer bestimmten Größe (hier bereits ab $k=21$) ist der Graph nicht mehr unbedingt zusammenhängend. Das liegt daran, dass aufgrund von Fehlern in den Daten sich nicht alle k -mere mit (min. 2) anderen überlappen und so ist es nicht möglich eine einzige, fortlaufende Sequenz zu rekonstruieren.

1.4 Viele Assembler basieren auf de Bruijn Graphen. Einer von ihnen ist velvet. Lies dir das zugehörige Paper¹ durch und beantworte folgende Fragen:

Wie unterscheidet sich der Velvet-Graph von einem gewöhnlichen De Bruijn (Sub-) Graphen? Velvet benutzt bidirektionale, kompakte De Bruijn (Sub-)Graphen.

Wie beeinflusst die Wahl der k -mer-Länge das Ergebnis des Assemblys? Die Wahl der k -mer Länge beeinflusst den Aufbau des Graphen. Dabei sorgt eine kleinere k -mer-Länge dafür, dass mehr Knoten vorliegen, mehr Überlappungen gefunden werden und der Graph somit mehr Kanten hat. Wenn generell mehr Überlappungen gefunden werden, steigt jedoch zwangsläufig die Anzahl irrtümlich identifizierter Überlappungen und damit sinkt die Spezifität des Assemblys und es kann zu Fehlern in der Rekonstruktion der Sequenz kommen.

Eine größere k -mer Länge reduziert die Anzahl an Knoten und es werden weniger Überlappungen gefunden. Bei großen k -mer-Längen und niedriger Abdeckung kann es im ungünstigsten Fall zu zusammenhangslosen, und somit nutzlosen, Graphen kommen. Bei weniger Überlappungen steigt jedoch die Spezifität und irrtümlich identifizierte Überlappungen werden unwahrscheinlicher.

Je nach Abdeckung muss entschieden werden, ob die Spezifität (lange k -mere) oder die Sensitivität gegenüber Überlappungen (kurze k -mere) priorisiert werden soll. Außerdem muss die k -mer Länge ungerade sein, damit ein k -mer nicht sein eigenes reverses Komplement sein kann und echt kleiner als die Read-Länge, damit überlappungen gefunden werden können. Es kann jedoch auch bereits bei echt kleineren k -mer-Längen der Fall sein, dass keine Überlappungen mehr gefunden werden können:

Kann für k eine gerade Zahl gewählt werden? Warum?/Warum nicht? Nein, man muss eine ungerade Zahl wählen, damit k nicht sein eigenes reverses Komplement sein kann. ("To ensure that each k -mer cannot be its own reverse complement, k must be odd.", S. 822)

Tips und Bubbles sind zwei häufig beobachtete Strukturen in De Bruijn (Sub-) Graphen und verwandten Graphen. Finde je ein Beispiel dieser Strukturen in einer der Visualisierungen mit Bandage und bilde sie in deiner Abgabe ab. Im Paper steht zu Tips: "A tip is a chain of nodes that is disconnected on one end." Das ist bspw. in Abbildung 5 (mehrfach) und 6c zu sehen.

Bubbles bestehen aus zwei Pfaden mit ähnlichen Sequenzen, die in den selben Knoten beginnen und enden. Man kann davon ausgehen, dass sie aufgrund von fehlerhaften Daten entstanden sind und somit redundant.

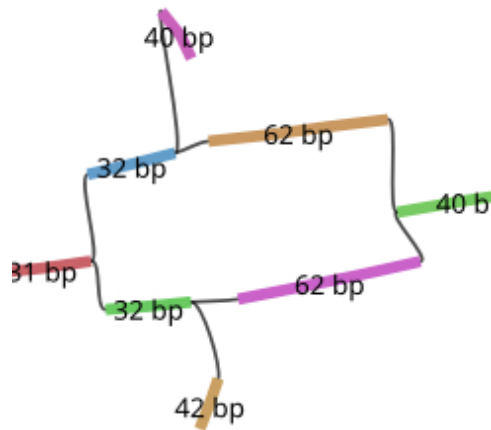


Figure 7: Mögliche Bubble.

In Abbildung 7 ist eine mögliche Bubble aus dem Graph für $k=31$ zu sehen. Ohne die Annotation der Sequenzen ist es nicht eindeutig zu sagen, aber da die Längenverteilung sehr ähnlich ist, kann man dies als Beispiel nehmen.

Beschreibe kurz, wie Velvet mit Tips und Bubbles umgeht. Bubbles werden mit dem sog. Tour-Bus-Algorithmus entfernt, wenn sie als fehlerhaft bzw. redundant befunden wurden. Dies wird auf Basis der Sequenzidentität und Länge bestimmt.

Wenn Tips als fehlerhaft eingestuft werden, weil zu vermuten ist, dass schlichtweg eine Lücke in der Abdeckung ist, dann werden sie nicht entfernt. Ansonsten werden sie einfach entfernt.

Tips sind fehlerhaft, wenn sie kürzer als 2000bp sind oder sie nur eine Alternative zu häufigeren Pfaden sind.

1.5 Informiere dich im Manual (z.B. zu finden unter `/vol/biotools/share/velvet/`) darüber, was die Aufgaben von `velveth` und `velvetg` sind und nenne sie in deiner Abgabe. Mache dich außerdem mit der einfachen Syntax der beiden Programme bekannt.

Velvet ist ein aus zwei Programmteilen bestehender Algorithmus für "de novo" Sequence Assembly und wird über Eingaben in der Kommandozeile genutzt. Der zuerst angewendete Teil namens `velveth` dient dazu, die zugeführten Sequenz-Daten aufzubereiten, damit sie anschließend durch den zweiten Programmteil `velvetg` verwendet werden können. Dies geschieht durch die Erstellung von Hash-Tabellen aus den Sequenz-Daten und darauf basierend automatisch erstellten Output-Dateien, die die Sequenzen und eine Roadmap enthalten.

Durch einen Aufruf des zweiten Teilprogramms `velvetg` werden aus den vorbereiteten Daten de Bruijn-Graphen erstellt und diese vereinfacht und korrigiert. Anschließend werden alle Contigs länger als das Zweifache der gesetzten Wortlänge k in einer Datei ausgegeben und weitere Dateien mit Informationen über die Graphen erstellt. In der Anwendung muss darauf geachtet

werden, dass der genannte Ausgabeordner mit dem Ordner übereinstimmt, in dem vorher die Output-Dateien von velvet gespeichert wurden.

1.6 Teste Velvet mit $k = 11, 15, 17, 21, 31$ auf den Reads. Nutze dazu einen Coverage-Cutoff von 3

Gib die verwendeten Kommandozeilenbefehle für velvetg und velveth auch in deiner Abgabe an: Durch die Verwendung des folgenden Befehls haben wir die kurzen Reads aus der Datei test reads bezüglich verschiedener k-mer-Länge verarbeiten lassen.

```
velveth . 11 -fastq -short reads.fq
```

(Mit den verschiedenen Zahlen natürlich)

```
velvetg . -cov_cutoff 3
```

1.7 Vergleiche anschließend die Ergebnisse.

Wie viele Knoten haben deine Graphen und wie unterscheiden sich die n50-Werte?

Bei großen oder kleinen k sind die n50-Werte niedriger als bei mittleren k und es gibt auch mehr Knoten im Graph (siehe Tabelle).

k	11	15	17	21	31
nodes	112	3	1	2	16
n50	107	2994	5297	5046	270
max	312	2994	5297	5046	398
total	5209	5298	5297	6265	3336

Erkläre, wie die Ergebnisse zustande gekommen sind. DFas liegt daran, dass bei kürzeren k-mer-Längen deutlich mehr Überlappungen gefunden werden und dadurch mehr Knoten in den Graphen vorliegen, als bei längeren k-meren. Dass der n50 und die Anzahl der Knoten zusammen hängen ist zu erwarten, da der n50-Wert die Qualität des Assemblies beschreibt und große Anzahlen von Knoten auf fehlerhaft identifizierte Überlappungen hinweisen können.

1.8 Wähle ein Assembly mit möglichst wenigen Contigs und vergleiche die assemblierte(n) Contig-sequenz(en) mit der Referenzsequenz reference.fasta.

Dafür nehmen wir die Assembly mit $k=17$.

Aligniere die beiden Sequenzen, z.B. mit BLAST Wir haben die BLAST Typ blastn auf der Seite des NCBI genutzt. Query war die Assembly und Subject die gestellte Referenz (siehe Abb. 8).

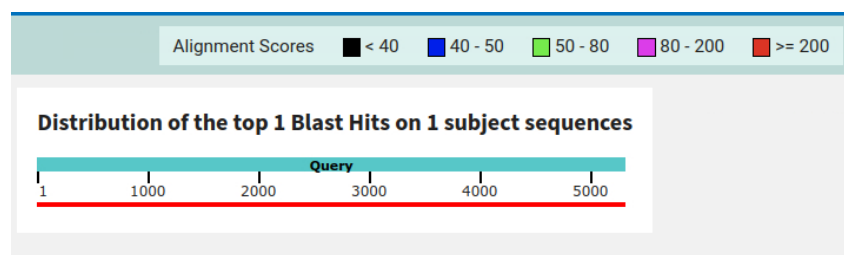


Figure 8

Diskutiere kurz dein Ergebnis Es gibt 100%iges Query-Cover und 99.92% Übereinstimmung Identität, also ist die Assembly komplett in der Referenz enthalten, aber die Referenz ist etwas länger. Die Assembly ist also nicht perfekt, aber nah dran.

Erstelle einen (compacted bi-directed) De Bruijn (Sub-) Graphen der beiden Sequenzen mit ggcats und visualisiere ihn mit Bandage. Nutze dabei ein eher größeres k. Beide Dateien zusammen in eine kombiniert:

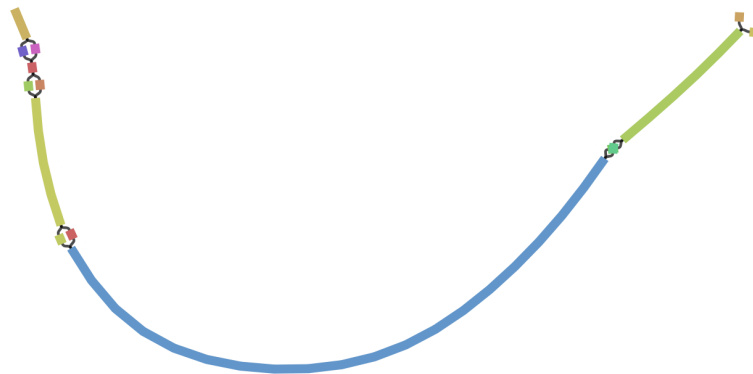
```
cat reference.fasta k17/contigs.fa > combi.fa
```

Dann mit ggcats erstellt:

```
ggcats build -gfa-v1 -kmer-length 21 -s 1 -e combi.fa -o out_combi.gfa
```

Wie sehen die Unterschiede im Graphen aus? Die Unterschiede sind sehr gering.

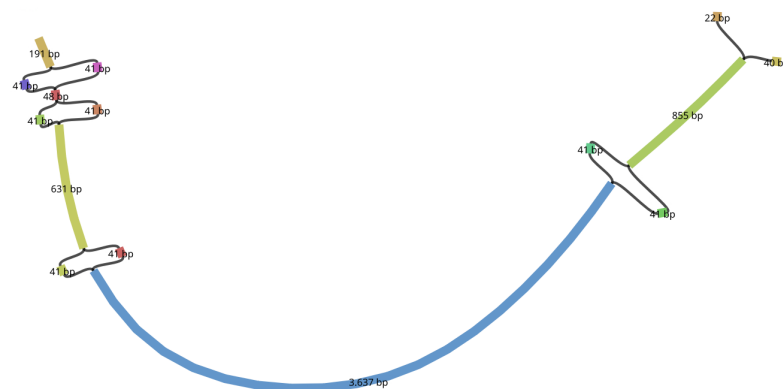
Bandage - /home/sant/seqan2/out_combi.gfa



(a) Graph wie von Bandage erstellt.

Bandage - /home/sant/seqan2/out_combi.gfa

help



(b) Graph mit Annotation der Längen und visuelle entzerrt.

Figure 9

Man kann sehen, dass die Unterschiede an einer begrenzten Anzahl von Stellen ist, über kurze Abschnitte geht und man kann vermuten, dass es sich dabei um Bubbles handelt.

References

- [1] algbio. *GGCAT - compacted and colored de Bruijn graph construction and querying*. <https://github.com/algbio/ggcat>. GitHub repository. n.d.
- [2] P. J. A. Cock et al. “The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants”. In: *Nucleic Acids Research* 38.6 (Apr. 2010). Epub 2009 Dec 16. PMID: 20015970; PMCID: PMC2847217, pp. 1767–1771. DOI: 10.1093/nar/gkp1137.
- [3] *FASTA and FASTQ formats*. Accessed 10 Mai 2026. URL: <https://compgenomr.github.io/book/fasta-and-fastq-formats.html>.